

Relatórios do sistema RUMO CERTO, que simula um portal de viagens moderno.

Relatório 1 via DevTools Lighthouse (antes das melhorias):



Desempenho: 75

Acessibilidade: 96

Práticas recomendadas: 100

SEO: 100

Gargalos encontrados:

1 Renderizar solicitações de bloqueio

As solicitações estão bloqueando a renderização inicial da página, o que pode atrasar a LCP. Essas solicitações de rede podem ser deferidas ou colocadas inline para que fiquem fora do caminho crítico.

2 Árvore de dependência da rede

Evite encadear solicitações críticas reduzindo o tamanho das cadeias, o tamanho do download de recursos ou adiando o download de recursos desnecessários para melhorar o carregamento da página.

3. JavaScript legado

Polyfills e transformações permitem que navegadores mais antigos usem novos recursos do JavaScript. No entanto, muitos não são necessários para navegadores mais recentes. Considere mudar seu processo de criação do JavaScript para não transpilar os recursos de referência, a menos que você precise da compatibilidade com navegadores mais antigos.

4. Detalhamento da LCP

Cada subparte tem estratégias de melhoria específicas. O ideal é que a maior parte do tempo de LCP seja gasto no carregamento dos recursos, não em atrasos.

5. Terceiros

Código de terceiros pode afetar significativamente a performance de carregamento. Reduza e adie o carregamento de código de terceiros para priorizar o conteúdo da sua página.

Diagnósticos:

Reduza o JavaScript :A redução de arquivos JavaScript pode diminuir o tamanho de payloads e o tempo de análise de scripts

A página impede a restauração do cache de avanço e retorno. Muitas navegações voltam para uma página anterior ou avançam de novo. O cache de avanço e retorno (bfcache) pode acelerar essas navegações de retorno

Reduza o JavaScript não usado: Para diminuir o consumo de bytes da atividade da rede, reduza o JavaScript não usado e adie o carregamento de scripts até que eles sejam necessários

Evitar tarefas longas da linha de execução principal :Lista as tarefas mais longas na linha de execução principal. Útil para identificar os piores contribuidores para a latência de entrada

Contraste:

As cores de primeiro e segundo plano não têm uma taxa de contraste suficiente.

Relatório 2 via DevTools Lighthouse (após as melhorias):



Desempenho: 100

Acessibilidade: 100

Práticas recomendadas: 100

SEO: 100

Otimizações Aplicadas:

ESLint: problemas resolvidos (erros de links internos com `<a>` e avisos de variáveis/imports não usados). Estado atual: sem problemas reportados.

Código morto: removido componente não utilizado `DeferredSection`.

Navegação: migração de âncoras internas para `next/link` em componentes e páginas.

Imagens: conversão de `<img>` para `next/image` em lista e detalhe de destinos.

Build: produção gerada com sucesso e minificação automática (Next.js/SwC).

Técnicas que trouxeram maior impacto:

Imagens (LCP): uso de `next/image` para otimização automática; manter assets fotográficos em **WebP/AVIF**, redimensionar para o tamanho real de exibição e usar `priority` apenas para heróis; listas e conteúdos abaixo da dobra mantêm lazy-load padrão.

HTML/CSS/JS: minificação e tree-shaking via build de produção do Next (`npm run build`), sem plugins adicionais.

Código: remoção de trechos não utilizados (imports/variáveis e componente sem referência) reduz bundle e evita custo de renderização.

Imports: preferência por recursos nativos do Next/React (ex.: `next/link`, `next/image`) em vez de bibliotecas externas pesadas.

Diretrizes práticas:

Imagens: para fotos, prefira **webp** ou **avif** em `public/`, definindo dimensões compatíveis com o layout. Use `priority` apenas em imagens críticas (herói) e mantenha lazy nas demais.

Estilos: remova classes não utilizadas dos CSS Modules para reduzir o CSS final.

Código: mantenha o projeto com ESLint ativo para detectar imports/variáveis não usados.

Imports: evite libs grandes para utilidades simples; prefira APIs nativas do browser/React/Next.